Docker y Docker Compose en el laboratorio Hadoop

El clúster del curso por dentro



Curso 2026/2027

Diego Sevilla
Departamento de Ingeniería y Tecnología de Computadores
Facultad de Informática · Universidad de Murcia

dsevilla@um.es

Para qué sirven estas transparencias

El clúster del curso son **contenedores Docker** que se arrancan y paran con **Docker Compose** y con el `Makefile` de `entorno/`

- No hay que construir las imágenes: se descargan ya publicadas
- Sí hay que **entender** cómo están hechas y qué guarda cada pieza
- Y saber **mirar el sistema** en cualquier momento: qué está en marcha, dónde están los datos, qué se pierde con cada orden de parada

> Todos los ejemplos usan los ficheros reales del curso: `make-images/Dockerfile*`, `entorno/compose-*.yml` y `entorno/Makefile`

Dónde se ejecuta cada orden

En estas transparencias cada bloque indica dónde se teclea:

Comentario	Dónde	Ejemplo
`# equipo`	Terminal de tu equipo (en Windows, la de WSL2), en la raíz de la distribución: el directorio que contiene `s1/` y `entorno/`	`make -C entorno status`
`# namenode`	Dentro del contenedor, tras `docker exec -it namenode bash`	`su - luser`

- `docker` y `make` sólo existen en tu equipo
- `hdfs`, `yarn` o `java` sólo existen dentro de los contenedores

Índice

- **Contenedores e imágenes**: qué es cada cosa, capas, nombres y registros
- **Órdenes básicas**: `pull`, `run`, `ps`, `exec`, `logs`, `cp`, `rm`
- **Dockerfile**: cómo se construyen `hadoop-base`, `namenode-image` y `datanode-image`
- **Volúmenes**: dónde vive el HDFS y qué sobrevive a cada orden
- **Redes**: `hadoop-cluster`, nombres de *host* y puertos publicados
- **Docker Compose**: `compose-hadoop-cluster.yml` línea a línea
- **El `Makefile` del entorno**: arrancar, parar y borrar
- **Ver cómo está el sistema**: diagnóstico paso a paso
- **Actividades**

Contenedores e imágenes

Qué es cada pieza y cómo se relacionan

Qué es un contenedor

- Un **proceso** (o grupo de procesos) del sistema anfitrión que el núcleo aísla del resto:
  - *namespaces*: lista de procesos, red, sistema de ficheros, nombre de *host* y usuarios propios
  - *cgroups*: límites de CPU y memoria (`cpus: 2.0`, `mem_limit: 4096m`)
- **No es una máquina virtual**: no hay otro núcleo ni se arranca un sistema operativo; comparte el núcleo Linux del anfitrión
- En Windows y macOS, Docker Desktop ejecuta una **máquina virtual Linux ligera** (WSL2 en Windows) y los contenedores viven dentro de ella

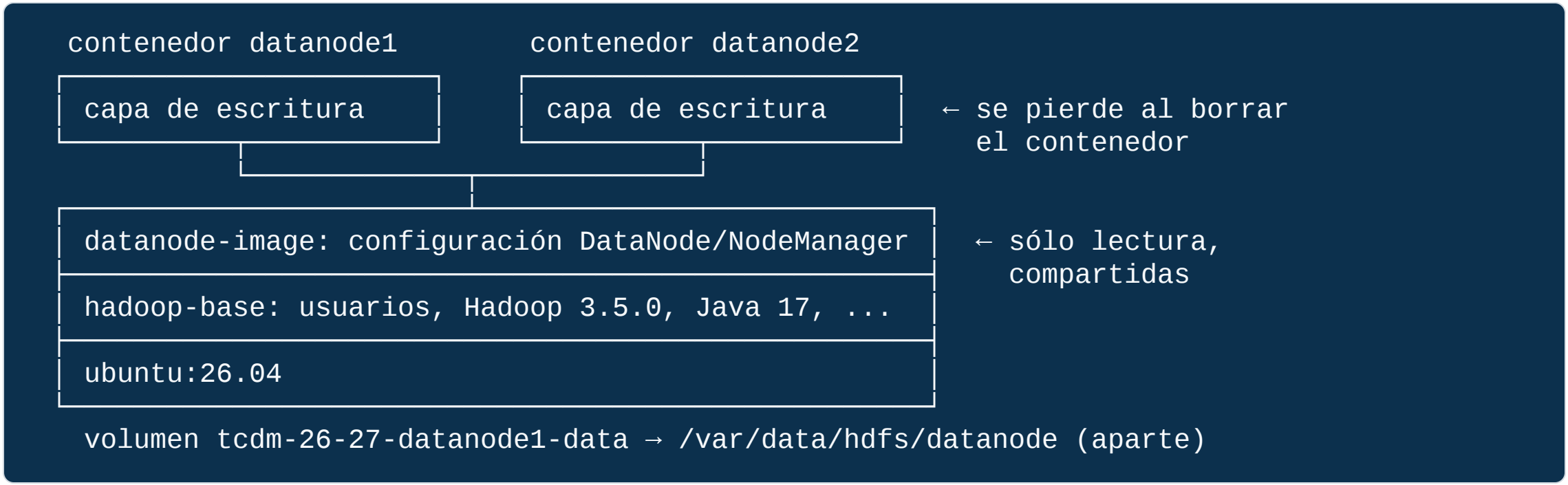
> Por eso la memoria que asignas a Docker Desktop es el techo de **todo** el clúster: los cuatro contenedores comparten esa máquina virtual

Las cuatro piezas de Docker

Pieza	Qué es	En el curso
Imagen	Sistema de ficheros de sólo lectura, por capas, más metadatos (variables, orden inicial, puertos)	`dsevilla/namenode-image:26-27`, `dsevilla/datanode-image:26-27`
Contenedor	Una imagen en ejecución, con una **capa de escritura** propia encima	`namenode`, `datanode1`, `datanode2`, `datanode3`
Volumen	Almacenamiento gestionado por Docker que **no pertenece** al contenedor y sobrevive a su borrado	`tcdm-26-27-namenode-data`, `tcdm-26-27-datanodeN-data`
Red	Red virtual con DNS propio entre contenedores	`hadoop-cluster`

- De una misma imagen salen muchos contenedores: `datanode1`, `datanode2` y `datanode3` comparten `datanode-image`
- Borrar un contenedor no borra su imagen ni sus volúmenes

Capas y copia en escritura



- Al modificar un fichero de la imagen se copia primero a la capa de escritura
- Las capas comunes se descargan y se guardan **una sola vez** en el disco

Nombres de imagen y registros

```
docker.io / dsevilla / namenode-image : 26-27
└registro┘  └usuario┘  └──repositorio──┘ └etiqueta┘
```

- Si no se indica, el registro es Docker Hub (`docker.io`)
- Si no se indica, la etiqueta es `latest` : el curso **no** la usa para sus imágenes, porque `latest` cambia sin avisar; `26-27` identifica el curso
- Una etiqueta puede apuntar a varias imágenes, **una por arquitectura**:

```
# equipo
docker manifest inspect dsevilla/namenode-image:26-27 | grep architecture
#            "architecture": "amd64",
#            "architecture": "arm64",
```

Docker descarga sola la variante de tu equipo (PC con Intel/AMD o Mac con Apple Silicon)

Órdenes básicas

Imágenes y contenedores desde la terminal

Imágenes: descargar, listar, inspeccionar y borrar

```
# equipo
docker pull dsevilla/datanode-image:26-27     # descarga (Compose lo hace solo)
docker image ls                               # imágenes locales y su tamaño
docker image inspect dsevilla/datanode-image:26-27 \
    --format '{{.Config.Entrypoint}} {{.Config.Cmd}}'
# [/usr/bin/tini -g --] [/inicio.sh]
docker image history dsevilla/namenode-image:26-27   # capas y su tamaño
docker image rm dsevilla/datanode-image:26-27        # falla si un contenedor la usa
```

- `inspect` devuelve JSON; `--format` extrae un campo con plantillas de Go
- `history` muestra cada instrucción del Dockerfile y cuánto ocupa su capa: la del entorno Python del NameNode es la mayor

`docker run`: crear y arrancar un contenedor

`docker run [opciones] imagen [orden]` — la orden, si se da, sustituye a la `CMD` de la imagen

Opción	Efecto
`-d`	Segundo plano (*detached*)
`-i -t` (`-it`)	Entrada estándar abierta y terminal: para un `bash` interactivo
`--rm`	Borra el contenedor al terminar (si no, se acumulan contenedores parados)
`--name nombre`	Nombre fijo en lugar de uno aleatorio
`-p 127.0.0.1:9870:9870`	Publica un puerto del contenedor en el equipo, sólo en la interfaz local
`-v volumen:/ruta`	Monta un volumen con nombre (o un directorio del equipo)
`-e VAR=valor`	Variable de entorno
`--network`, `--memory`, `--cpus`	Red a la que se conecta; límites de memoria y CPU

El clúster no se lanza con `docker run`: Compose reúne todas estas opciones en un fichero

Mirar una imagen sin arrancar el clúster

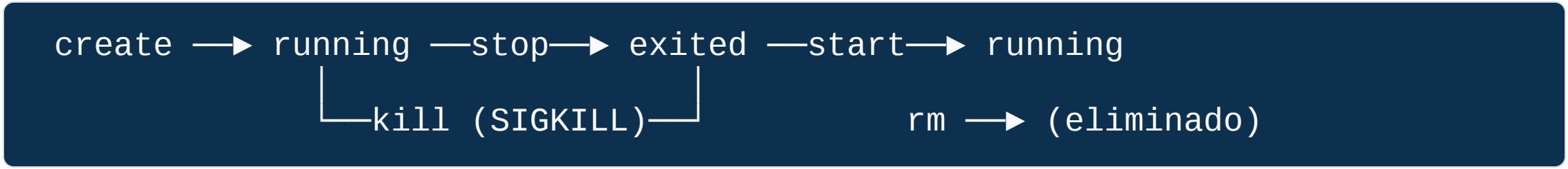
Un contenedor de usar y tirar sirve para explorar la imagen:

```
# equipo
docker run --rm -it dsevilla/datanode-image:26-27 bash
```

```
# dentro del contenedor temporal (sale con exit y se borra)
cat /inicio.sh                                  # qué haría al arrancar
ls /opt/bd/hadoop/etc/hadoop/                   # configuración de Hadoop
grep -A1 resource.memory-mb /opt/bd/hadoop/etc/hadoop/yarn-site.xml
su - hdadmin -c jps                             # no hay demonios: sólo Jps
```

- `bash` sustituye a `/inicio.sh`: no arranca ningún demonio
- No se conecta a `hadoop-cluster` ni monta volúmenes: no toca tu clúster

Ciclo de vida de un contenedor



Orden	Efecto	Capa de escritura
`docker stop namenode`	Envía `SIGTERM`; a los `10` s, `SIGKILL`	Se conserva
`docker start namenode`	Vuelve a ejecutar la orden inicial en el **mismo** contenedor	Se conserva
`docker restart namenode`	`stop` + `start`	Se conserva
`docker rm namenode`	Elimina un contenedor parado	**Se pierde**
`docker ps` / `docker ps -a`	En marcha / también los parados	—

Código de salida en `STATUS`: `Exited (0)` normal, `(143)` terminado con `SIGTERM`, `(137)` con `SIGKILL`, a menudo **por falta de memoria**

Ejecutar órdenes dentro de un contenedor: `docker exec`

```
# equipo: una orden suelta, como en las celdas de S1
docker exec namenode su - luser -c 'hdfs dfs -ls /'

# equipo: una terminal interactiva
docker exec -it namenode bash
```

```
# namenode (eres root): cambia al usuario adecuado
su - hdadmin      # administra los demonios: hdfs dfsadmin -report, yarn node -list
su - luser        # trabaja con los datos: hdfs dfs -ls /datalake
```

- `docker exec` sólo funciona con el contenedor **en marcha**
- `su -` abre una *shell* de *login*: carga `/etc/profile.d/hadoop.sh`, que añade `hdfs` y `yarn` al `PATH`. Con `docker exec -u luser namenode hdfs ...` ese `PATH` no se carga y la orden no se encuentra

Copiar ficheros y leer la salida de un contenedor

```
# equipo: copiar entre el equipo y un contenedor (en ambos sentidos)
docker cp namenode:/opt/bd/hadoop/etc/hadoop/hdfs-site.xml .
docker cp mis-datos.csv namenode:/tmp/

# equipo: salida estándar del proceso inicial
docker logs namenode
docker logs --tail 50 -f datanode2        # últimas 50 líneas y seguir
```

- `docker logs` sólo muestra lo que escribe el proceso inicial (`/inicio.sh`): si un demonio no arranca, el *script* termina con error y aquí se ve por qué
- Los demonios de Hadoop escriben **sus propios** registros dentro del contenedor, en `/opt/bd/hadoop/logs/` (se verá en «Ver cómo está el sistema»)

Dockerfile

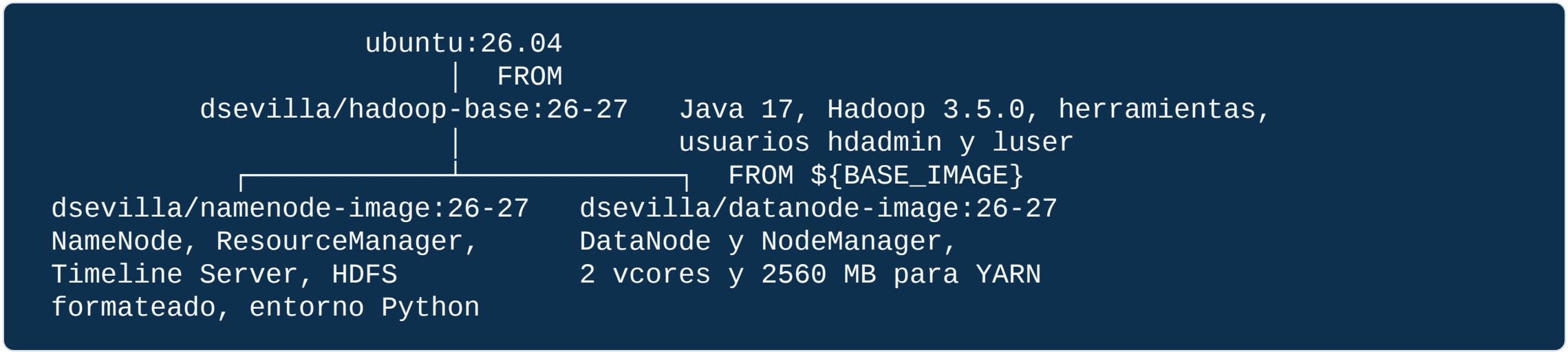
Cómo se construyen las imágenes del curso

Instrucciones de un Dockerfile

Instrucción	Qué hace
`FROM imagen`	Imagen de partida; cada `FROM` empieza una etapa
`ARG NOMBRE=valor`	Parámetro de construcción (`--build-arg`); no queda en el contenedor
`ENV NOMBRE=valor`	Variable de entorno que **sí** queda en la imagen y en sus contenedores
`COPY origen destino`	Copia ficheros del contexto de construcción a la imagen
`ADD url destino`	Como `COPY`, y además descarga URL (con `--checksum` para verificarlas)
`RUN orden`	Ejecuta una orden durante la construcción; el resultado es una capa nueva
`USER`, `WORKDIR`	Usuario y directorio para las instrucciones siguientes y el contenedor
`EXPOSE puertos`	**Documenta** los puertos del servicio; no publica nada
`ENTRYPOINT [...]`	Programa que se ejecuta siempre al arrancar
`CMD [...]`	Argumentos por defecto del `ENTRYPOINT`; `docker run imagen orden` los sustituye
`LABEL clave=valor`	Metadatos (autor, versión…)

Referencia: https://docs.docker.com/reference/dockerfile/

Tres imágenes relacionadas



- Hadoop se descarga e instala **una vez**, en la base
- Cada imagen de servicio añade sólo lo propio de su tipo de nodo
- `make-images/` no se distribuye con las sesiones: S1 («Cómo se construyen las imágenes del curso») y estas transparencias citan sus ficheros, y el resultado se puede inspeccionar en las propias imágenes

La imagen base: `make-images/Dockerfile`

```
FROM ubuntu:26.04

ARG HADOOP_VERSION=3.5.0
ENV HADOOP_VERSION=${HADOOP_VERSION} \
    JAVA_HOME=/usr/lib/jvm/java-17-openjdk \
    HADOOP_HOME=/opt/bd/hadoop \
    HADOOP_CONF_DIR=/opt/bd/hadoop/etc/hadoop

COPY install-base-packages.sh /root/install-base-packages.sh
RUN /bin/sh -x /root/install-base-packages.sh && \
    rm -f /root/install-base-packages.sh
# ... igual con install-hadoop.sh y configure-hadoop-users.sh

ENTRYPOINT ["/usr/bin/tini", "-g", "--"]
CMD ["/bin/bash"]
```

- `ARG` elige la versión al construir; `ENV` deja las rutas para siempre
- Cada paso es un *script* que se copia, se ejecuta y se borra

Un `RUN` por paso, con su limpieza dentro

`install-base-packages.sh` instala y limpia en **la misma** orden `RUN`:

```
apt-get update -qq
apt-get install -qq -y --no-install-recommends ca-certificates curl tini \
    openjdk-17-jdk-headless python3 python3-venv ...
apt-get clean
rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/* /tmp/* /var/tmp/*
```

- Cada `RUN` produce una capa, y una capa **no puede encoger** a las anteriores: borrar en un `RUN` posterior sólo oculta los ficheros
- Por eso la caché de `apt` se borra antes de que termine el mismo `RUN`
- `install-hadoop.sh` sigue el mismo criterio: descarga el `.tar.gz`, comprueba su SHA-512, lo descomprime y lo borra

Las imágenes de servicio: `Dockerfile.namenode`

```
ARG BASE_IMAGE=dsevilla/hadoop-base:26-27
FROM ${BASE_IMAGE}

COPY namenode.sh /tmp/
RUN sh /tmp/namenode.sh && \
    rm -f /tmp/namenode.sh

COPY requirements-namenode.txt install-python-environment.sh /tmp/
RUN sh /tmp/install-python-environment.sh && \
    rm -f /tmp/install-python-environment.sh /tmp/requirements-namenode.txt

EXPOSE 9870 9871 8088 8000-10000
CMD ["/inicio.sh"]
```

- `ARG` antes de `FROM` permite cambiar la imagen de partida al construir
- No repite `ENTRYPOINT`: hereda el `tini` de la base y sólo cambia `CMD`
- `Dockerfile.datanode` es igual, con `datanode.sh` y sin entorno Python

Qué hace `namenode.sh` durante la construcción

1. Escribe `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml` y `yarn-site.xml` con `cat > fichero <<'EOF'`
2. **Formatea** HDFS: `hdfs namenode -format` en `/var/data/hdfs/namenode`
3. Arranca un NameNode temporal y crea `/user/luser`, `/tmp`, `/datalake/{raw,silver,gold}` y `/warehouse` con sus permisos
4. Para el NameNode y genera `/inicio.sh`

> La imagen ya contiene un HDFS recién formateado y con sus directorios

- Para arrancar ese NameNode, `fs.defaultFS` (`hdfs://namenode:9000/`) debe resolverse: `createimages.sh` construye con `--add-host namenode:127.0.0.1`

Qué ocurre al arrancar: `ENTRYPOINT` + `CMD`

```
PID 1: /usr/bin/tini -g -- /inicio.sh
```

```
#!/usr/bin/env sh            # /inicio.sh del NameNode (resumido)
set -eu
su - hdadmin -c "... hdfs --daemon start namenode"
su - hdadmin -c "... yarn --daemon start resourcemanager"
su - hdadmin -c "... yarn --daemon start timelineserver"
exec tail -f /dev/null       # mantiene vivo el contenedor
```

- `tini` reenvía señales y recoge los procesos que quedan huérfanos
- Los demonios se lanzan en segundo plano; el contenedor vive mientras viva `tail`
- **Consecuencia**: `running` sólo significa que `tail` sigue vivo. Si el NameNode cae después, el contenedor sigue `running`; hay que comprobar Hadoop (`jps`, `dfsadmin -report`)

Construir: `docker build`

`make-images/createimages.sh` (construcción local, sin publicar):

```
docker build \
    --build-arg "BASE_IMAGE=${BASE_IMAGE}" \
    --add-host namenode:127.0.0.1 \
    -f Dockerfile.namenode \
    -t "dsevilla/namenode-image:${IMAGE_YEAR}" \
    .
```

- `.` es el **contexto**: el directorio cuyos ficheros puede usar `COPY`
- `-f` elige el Dockerfile; `-t` da nombre y etiqueta a la imagen
- El CI usa `docker buildx build --platform linux/amd64,linux/arm64 --push` (`buildx-createimages.sh`): construye las dos arquitecturas y las publica bajo la misma etiqueta. `install-hadoop.sh` elige el `.tar.gz` de Hadoop según la arquitectura

La caché de construcción

Docker reutiliza una capa si la instrucción y los ficheros que copia no han cambiado, **y** todas las capas anteriores tampoco

```
FROM ${BASE_IMAGE}                              ✔ caché
COPY namenode.sh /tmp/                          ✔ caché (namenode.sh igual)
RUN sh /tmp/namenode.sh ...                     ✔ caché
COPY requirements-namenode.txt ... /tmp/        ✘ cambió requirements-namenode.txt
RUN sh /tmp/install-python-environment.sh       ✘ se repite
EXPOSE / CMD                                    ✘ se repiten (instantáneas)
```

- Lo que cambia a menudo va **al final** del Dockerfile
- Cambiar una línea de `requirements-namenode.txt` no vuelve a formatear HDFS
- Cambiar la base obliga a reconstruir las dos imágenes de servicio

Construcción en varias etapas (desde S6)

`entorno/hive-metastore/Dockerfile`, que llega con la sesión 6:

```
ARG HIVE_IMAGE=apache/hive:4.2.1

FROM scratch AS postgresql-jdbc
ARG POSTGRES_JDBC_VERSION=42.7.13
ADD --checksum=sha256:6e0e4cc2...93 \
    https://repo1.maven.org/maven2/org/postgresql/postgresql/.../postgresql-${POSTGRES_JDBC_VERSION}.jar \
    /postgresql.jar

FROM ${HIVE_IMAGE}
USER root
COPY --from=postgresql-jdbc --chown=hive:hive /postgresql.jar /opt/hive/lib/postgres.jar
COPY --chmod=755 entrypoint-tcdm.sh /entrypoint-tcdm.sh
USER hive
ENTRYPOINT ["/entrypoint-tcdm.sh"]
```

- La primera etapa (`FROM scratch`, vacía) sólo descarga y verifica el `.jar`
- La imagen final copia de ella **únicamente** ese fichero (`COPY --from`)
- Compose pasa otra `HIVE_IMAGE` con `build.args` (se verá más adelante)

Volúmenes

Dónde viven los datos y qué sobrevive a cada orden

Tres sitios donde puede escribir un contenedor

Sitio	Ejemplo en el curso	Sobrevive a `docker compose down`
Capa de escritura del contenedor	`/tmp`, `/opt/bd/hadoop/logs`, almacén del Timeline Server en `namenode`	**No**
Volumen con nombre	`tcdm-26-27-namenode-data` en `/var/data/hdfs/namenode`	Sí
Directorio del equipo (*bind mount*)	`./trino-hdfs/etc` en `/etc/trino` (S6)	Sí: es un directorio de tu equipo

- Volumen: Docker decide dónde guardarlo; se gestiona con `docker volume`
- *Bind mount*: lo que cambias en el equipo se ve al instante dentro (y al revés, salvo con `:ro`, de sólo lectura)
- El curso guarda **todo el estado de HDFS en volúmenes**: metadatos del NameNode y bloques de cada DataNode

Los volúmenes del clúster Hadoop

```yaml
services:
  namenode:
    volumes:
      - namenode-data:/var/data/hdfs/namenode      # dfs.namenode.name.dir
  datanode1:
    volumes:
      - datanode1-data:/var/data/hdfs/datanode     # dfs.datanode.data.dir
  # datanode2 y datanode3: cada uno con su propio volumen
volumes:
  namenode-data:
    name: tcdm-26-27-namenode-data
  datanode1-data:
    name: tcdm-26-27-datanode1-data
  # ...
```

- A la izquierda de `:` el nombre corto del Compose; `name:` fija el nombre real en Docker, sin el prefijo del proyecto que Compose añadiría
- Cada DataNode tiene **su** volumen: no comparten bloques

El volumen se rellena con el contenido de la imagen

Al montar por primera vez un volumen **vacío** donde la imagen ya tiene ficheros, Docker copia esos ficheros al volumen

1. Primer `up`: `tcdm-26-27-namenode-data` no existe → se crea y recibe el HDFS formateado que dejó `namenode.sh` en la imagen
2. Siguientes `up`: el volumen ya tiene datos → se monta tal cual, con todo lo que hayas guardado en HDFS
3. Tras borrar los volúmenes (`clean-hdfs`): el próximo `up` vuelve al punto 1, un HDFS vacío como el del primer día

> Una imagen nueva **no** cambia un volumen con datos: conserva tu HDFS. Los cuatro volúmenes forman un todo: no se borran por separado

Inspeccionar los volúmenes

```
# equipo
docker volume ls --filter name=tcdm-26-27
docker volume inspect tcdm-26-27-namenode-data
docker system df -v | grep tcdm-26-27      # espacio que ocupa cada volumen
```

El `Mountpoint` que muestra `inspect` (`/var/lib/docker/volumes/...`) está dentro de la máquina virtual de Docker Desktop: para mirar dentro, se monta el volumen en un contenedor temporal, **de sólo lectura**:

```
# equipo
docker run --rm -v tcdm-26-27-namenode-data:/nn:ro \
    dsevilla/namenode-image:26-27 ls -l /nn/current
docker run --rm -v tcdm-26-27-datanode1-data:/dn:ro \
    dsevilla/namenode-image:26-27 sh -c "find /dn -name 'blk_*' | head"
```

Qué hay dentro de los volúmenes HDFS

- **NameNode** (`/var/data/hdfs/namenode/current`):
  - `fsimage_*` : imagen del espacio de nombres (directorios, ficheros, bloques de cada fichero)
  - `edits_*` : cambios posteriores a la última imagen
  - `VERSION` : incluye el `clusterID` del formateo
- **DataNode** (`/var/data/hdfs/datanode/current/BP-*/current/finalized/`):
  - `blk_<id>` y `blk_<id>_<gen>.meta` : cada bloque es un fichero normal (hasta 64 MB, `dfs.blocksize`) con sus sumas de control
  - `VERSION` con el **mismo** `clusterID` : un DataNode con otro `clusterID` se niega a unirse (`Incompatible clusterIDs` en su registro)

> Un mismo bloque aparece en tres volúmenes de DataNode: es la replicación 3

Copia de seguridad de HDFS

El contenido del clúster lo construyes tú sesión a sesión. Se puede guardar con el clúster **parado** (`make -C entorno stop`) y los cuatro volúmenes a la vez:

```
# equipo: un .tgz por volumen en el directorio actual
for v in namenode datanode1 datanode2 datanode3; do
  docker run --rm -v "tcdm-26-27-$v-data:/datos:ro" -v "$PWD:/copia" \
    dsevilla/namenode-image:26-27 tar -czf "/copia/$v.tgz" -C /datos .
done
```

Para restaurarla, con los volúmenes borrados y **antes** de `hadoop-up` :

```
# equipo
for v in namenode datanode1 datanode2 datanode3; do
  docker run --rm -v "tcdm-26-27-$v-data:/datos" -v "$PWD:/copia:ro" \
    dsevilla/namenode-image:26-27 tar -xzf "/copia/$v.tgz" -C /datos
done
```

Compose avisará de que esos volúmenes no los creó él: es un aviso, no un error

Cuidado con las órdenes de limpieza globales

Las órdenes de limpieza de Docker no saben qué volúmenes son tuyos del curso:

Orden	Qué borra
`docker container prune`	Todos los contenedores **parados**: tras `make -C entorno stop`, también los del clúster (los volúmenes quedan)
`docker image prune -a`	Imágenes sin contenedor: obliga a descargar de nuevo
`docker volume prune -a`	**Todos** los volúmenes sin contenedor: tras un `down`, **el HDFS entero**
`docker system prune`	Contenedores parados, redes sin uso, caché de construcción

> Para el laboratorio usa las órdenes del `Makefile`, que dicen en cada caso qué se conserva

Redes

Cómo se encuentran los contenedores

La red `hadoop-cluster`

- Red de tipo *bridge* con **DNS interno**: cada contenedor resuelve a los demás por su nombre de servicio, de contenedor o de alias
- Se declara **externa**: la crea el `Makefile`, no el Compose

```yaml
networks:
  hadoop-cluster:
    external: true
    name: hadoop-cluster
```

- Así varios proyectos Compose se ven entre sí: el clúster Hadoop, RustFS (S3) y, más adelante, Hive Metastore y Trino (S6) se unen a la misma red sin estar en el mismo fichero
- `namenode` tiene además dos alias, `resourcemanager` y `timelineserver`, que son los nombres que usa `yarn-site.xml`

Ver la red y resolver nombres

```
# equipo: qué contenedores hay en la red y con qué dirección IP
docker network inspect hadoop-cluster \
  --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}{{"\n"}}{{end}}'

# equipo: resolver nombres desde un DataNode
docker exec datanode1 getent hosts namenode
docker exec datanode1 getent hosts resourcemanager   # la misma IP
docker exec datanode1 ping -c 2 datanode3
```

- Las direcciones IP cambian al recrear los contenedores; los nombres no. Por eso la configuración de Hadoop sólo usa nombres
- `127.0.0.1` dentro de un contenedor es **ese** contenedor, no tu equipo

Puertos internos y puertos publicados

```yaml
namenode:
  ports:
    - "127.0.0.1:9870:9870" # HDFS NameNode
    - "127.0.0.1:8088:8088" # YARN ResourceManager
    - "127.0.0.1:8188:8188" # YARN Timeline Server
    - "127.0.0.1:8888:8888" # Jupyter Lab de luser, arrancado a mano desde S2
    - "127.0.0.1:4040:4040" # Spark UI mientras haya una aplicación activa
```

- `IP:equipo:contenedor`: sólo tu equipo (`127.0.0.1`) llega a esos puertos; otros ordenadores de la red no
- Dentro de `hadoop-cluster` **todos** los puertos están accesibles sin publicarlos: `namenode:9000` (RPC de HDFS) o `datanode1:9866` (bloques)
- El `9000` del NameNode no se publica, y por eso no choca con el `9000` que publica RustFS en S3: en la red son `namenode:9000` y `rustfs:9000`
- `EXPOSE` del Dockerfile no publica nada; lo hace `ports:`

Docker Compose

Un fichero describe todo el clúster

Qué es Docker Compose

- Describe en **YAML** servicios, volúmenes y redes, y los gestiona como un **proyecto**; se invoca como `docker compose` (Compose v2)
- Un fichero por grupo de servicios en `entorno/`:

Fichero	Proyecto (`name:`)	Desde
`compose-hadoop-cluster.yml`	`tcdm-26-27-hadoop`	S1
`compose-s3.yml`	`tcdm-26-27-s3`	S3
`compose-warehouse-hdfs.yml`	`tcdm-26-27-warehouse-hdfs`	S6

> En Internet verás `docker-compose` (con guion) y `version: '2'`: son de Compose v1. Compose v2 ignora `version:` y avisa de que sobra

`compose-hadoop-cluster.yml`: el NameNode

```yaml
name: tcdm-26-27-hadoop

services:
  namenode:
    image: ${NAMENODE_IMAGE:-dsevilla/namenode-image:26-27}
    container_name: namenode
    hostname: namenode
    ports:
      - "127.0.0.1:9870:9870" # HDFS NameNode
      # ... 8088, 8188, 8888, 4040
    cpus: 2.0
    mem_limit: 4096m
    volumes:
      - namenode-data:/var/data/hdfs/namenode
    networks:
      hadoop-cluster:
        aliases:
          - resourcemanager
          - timelineserver
```

Las claves de un servicio

- `image`: imagen de partida. `${VAR:-valor}` usa la variable de entorno `NAMENODE_IMAGE` si existe y, si no, el valor por defecto
- `container_name`: nombre fijo del contenedor. Sin él sería `tcdm-26-27-hadoop-namenode-1`; con él, `docker exec namenode` funciona
- `hostname`: nombre del *host* dentro del contenedor (lo que ve Hadoop)
- `cpus`, `mem_limit`: límites del contenedor, **no** reservas. El NameNode aloja NameNode, ResourceManager, Timeline Server y, desde S2, Jupyter y el *driver* de Spark
- Cada DataNode: `cpus: 2.0` y `mem_limit: 3072m`, de los que YARN anuncia 2560 MB (`yarn-site.xml` de la imagen) y deja 512 MB para el DataNode, el NodeManager y el resto de procesos

Ver el Compose tal y como lo entiende Docker

```
# equipo: fichero final, con variables sustituidas y valores normalizados
docker compose -f entorno/compose-hadoop-cluster.yml config

# equipo: probar una variable sin arrancar nada
NAMENODE_IMAGE=prueba/otra:1 \
  docker compose -f entorno/compose-hadoop-cluster.yml config | grep image
```

```
    image: dsevilla/datanode-image:26-27
    image: dsevilla/datanode-image:26-27
    image: dsevilla/datanode-image:26-27
    image: prueba/otra:1
```

- `config` valida el fichero y no crea ni arranca nada
- Compose lee también un fichero `.env` del directorio del Compose (`entorno/.env.example` muestra las variables que se pueden cambiar)

Órdenes de Compose

Todas con `docker compose -f entorno/compose-hadoop-cluster.yml ...`

Orden	Efecto	Contenedores	Volúmenes
`up -d`	Descarga lo que falte, crea y arranca	Crea o reutiliza	Crea si no existen
`ps -a`	Estado de los servicios del proyecto	—	—
`logs -f [servicio]`	Salida de todos o de uno	—	—
`top`	Procesos de cada contenedor	—	—
`exec servicio orden`	Como `docker exec`, por nombre de servicio	—	—
`stop` / `start`	Para / reanuda los mismos contenedores	Se conservan	Se conservan
`down`	Para y **elimina** contenedores	Se pierden	Se conservan
`down -v`	Además borra los volúmenes **del fichero**	Se pierden	**Se pierden**
`pull`	Descarga versiones nuevas de las imágenes	—	—

`up -d` **recrea** un contenedor si cambió su imagen o su configuración: se pierde su capa de escritura, pero no sus volúmenes

Salud y dependencias: `compose-s3.yml`

```yaml
services:
  rustfs-s3:
    image: rustfs/rustfs:${RUSTFS_VERSION:-latest}
    volumes:
      - rustfs-s3-data:/data
    healthcheck:
      test: ["CMD-SHELL", "curl --fail http://127.0.0.1:9000/health && ..."]
      interval: 10s
  s3-client:
    image: amazon/aws-cli:latest
    depends_on:
      rustfs-s3:
        condition: service_healthy     # espera a que RustFS responda
    entrypoint: ["/bin/sh", "-c"]
    command: ["while :; do sleep 3600; done"]   # simplificado
```

- `healthcheck`: Docker ejecuta la prueba y marca `healthy` o `unhealthy`
- `s3-client` no hace nada: queda vivo para usar `aws` con `docker exec`

Construir desde Compose y servicios de un solo uso (S6)

Fragmentos de `compose-warehouse-hdfs.yml`:

```
  hive-metastore:
    build:
      context: ./hive-metastore           # directorio con el Dockerfile
      args:
        HIVE_IMAGE: apache/hive:${HIVE_VERSION:-3.1.3}
    image: tcdm/hive-metastore:${HIVE_VERSION:-3.1.3}   # nombre de la imagen
  trino-hdfs:
    volumes:
      - ./trino-hdfs/etc:/etc/trino:ro     # configuración desde tu equipo
  catalog-init:
    entrypoint: ["/bin/sh", "/opt/tcdm/init/initialize-catalogs.sh"]
    restart: "no"                          # se ejecuta una vez y termina
```

- `build` + `image`: Compose construye en tu equipo y etiqueta el resultado
- `catalog-init` aparecerá como `Exited (0)`: es lo esperado

El `Makefile` del entorno

Las órdenes de Compose, con nombre propio

Qué hace el `Makefile`

```
HADOOP_COMPOSE := docker compose -f compose-hadoop-cluster.yml

network:
	docker network inspect hadoop-cluster >/dev/null 2>&1 || docker network create hadoop-cluster

hadoop-up: network
	$(HADOOP_COMPOSE) up -d

s3-up: network
	$(S3_COMPOSE) up -d
```

- `make -C entorno objetivo` entra en `entorno/` y ejecuta el objetivo: por eso dentro del `Makefile` las rutas no llevan `entorno/`
- `hadoop-up: network` significa que antes se ejecuta `network`
- `network` es **idempotente**: crea la red sólo si no existe

Arrancar, parar y borrar: de menos a más destructiva

Orden (en el equipo)	Compose que ejecuta	Qué se conserva	Qué se pierde
`make -C entorno stop`	`stop`	Todo, incluido el interior de cada contenedor	Nada (Jupyter hay que lanzarlo de nuevo)
`make -C entorno down`	`down`	Volúmenes: HDFS, metadatos del Hive Metastore, objetos de S3	Capa de escritura de los contenedores (`/tmp` , registros, Timeline Server)
`make -C entorno clean`	`down -v` (salvo Hadoop: `down`)	Sólo HDFS	Hive Metastore y S3
`make -C entorno clean-hdfs`	`clean` + `docker volume rm` de los cuatro volúmenes HDFS	Nada	**Todos los datos**, incluido `/datalake`

- Después de cualquiera: `make -C entorno hadoop-up` (y `s3-up` si lo usas)
- `make -C entorno status` : `ps -a` de cada Compose presente
- `make -C entorno jupyter` : atajo de `docker exec -it namenode su - luser -c 'jupyter lab ...'`

Ver qué haría `make` sin ejecutarlo

`make -n` imprime las órdenes de un objetivo **sin ejecutarlas**:

```
# equipo
make -n -C entorno clean-hdfs
```

```
if test -e compose-trino.yml; then docker compose -f compose-trino.yml down -v; fi
if test -e compose-s3.yml; then docker compose -f compose-s3.yml down -v; fi
if test -e compose-warehouse-hdfs.yml; then docker compose -f ... down -v; fi
docker compose -f compose-hadoop-cluster.yml down
docker volume rm -f tcdm-26-27-namenode-data tcdm-26-27-datanode1-data ...
```

- El Compose de Hadoop se baja **sin** `-v` en `clean`: sus volúmenes sólo los borra `clean-hdfs`, de forma explícita
- Los `if test -e` permiten que el `Makefile` funcione aunque aún no tengas los Compose de sesiones posteriores

Ver cómo está el sistema

Diagnóstico de fuera hacia dentro

De fuera hacia dentro

1. **¿Responde Docker?** `docker info` (si no: arranca Docker Desktop o el servicio)
2. **¿Existen y están en marcha los contenedores?** `make -C entorno status`
3. **¿Qué consumen?** `docker stats --no-stream`
4. **¿Arrancó bien el proceso inicial?** `docker logs namenode`
5. **¿Están vivos los demonios?** `docker exec namenode su - hdadmin -c jps`
6. **¿Funciona Hadoop?** `hdfs dfsadmin -report`, `yarn node -list -all`
7. **¿Qué dicen los registros de Hadoop?** `/opt/bd/hadoop/logs/`

> Cada paso sólo tiene sentido si el anterior está bien: un `jps` vacío no se arregla repitiendo celdas de HDFS

Estado y consumo de los contenedores

```
# equipo
make -C entorno status
docker stats --no-stream namenode datanode1 datanode2 datanode3
docker compose -f entorno/compose-hadoop-cluster.yml top namenode
```

```
NAME         CPU %   MEM USAGE / LIMIT     MEM %
namenode     1.52%   1.31GiB / 4GiB        32.8%
datanode1    0.61%   612MiB / 3GiB         19.9%
...
```

(columnas y valores de ejemplo: dependen de lo que esté ejecutando el clúster)

- `LIMIT` es el `mem_limit` del Compose; si un contenedor lo supera, el núcleo mata un proceso y el contenedor puede acabar en `Exited (137)`
- `top` muestra `tini`, `tail` y una JVM por demonio de Hadoop

`docker inspect`: toda la configuración de un contenedor

```
# equipo
docker inspect namenode --format '{{.State.Status}} desde {{.State.StartedAt}}'
docker inspect namenode --format '{{.State.ExitCode}} OOM={{.State.OOMKilled}}'
docker inspect namenode --format '{{.HostConfig.NanoCpus}} {{.HostConfig.Memory}}'
# 2000000000 4294967296      ← cpus: 2.0 y mem_limit: 4096m
docker inspect namenode \
  --format '{{range .Mounts}}{{.Name}} -> {{.Destination}}{{"\n"}}{{end}}'
# tcdm-26-27-namenode-data -> /var/data/hdfs/namenode
```

- `OOMKilled=true` confirma que el contenedor se quedó sin memoria
- Sin `--format`, `inspect` muestra el JSON completo: `Config`, `State`, `Mounts`, `NetworkSettings`, `HostConfig`

Qué se perdería con un `down`: `docker diff`

`docker diff` lista lo que ha cambiado en la **capa de escritura** respecto a la imagen (`A` añadido, `C` cambiado, `D` borrado):

```
# equipo
docker diff namenode | head -20
```

```
C /opt/bd/hadoop-3.5.0/logs
A /opt/bd/hadoop-3.5.0/logs/hadoop-hdadmin-namenode-namenode.log
A /opt/bd/hadoop-3.5.0/logs/hadoop-hdadmin-resourcemanager-namenode.log
C /tmp
A /tmp/hadoop-hdadmin-namenode.pid
C /var/data/yarn/timeline
...
```

(salida abreviada; `/opt/bd/hadoop` es un enlace a `/opt/bd/hadoop-3.5.0`)

- No aparece `/var/data/hdfs/namenode`: es un volumen, no la capa de escritura
- Todo lo que sí aparece desaparece con `down` y se conserva con `stop`

Los registros de Hadoop

```
# equipo
docker exec namenode ls /opt/bd/hadoop/logs/
docker exec namenode sh -c 'tail -n 40 /opt/bd/hadoop/logs/*-namenode-*.log'
docker exec datanode2 sh -c 'grep -i -E "error|exception" /opt/bd/hadoop/logs/*-datanode-*.log | tail'
```

- Un fichero por demonio y contenedor: `hadoop-<usuario>-<demonio>-<host>.log`
- Viven en la capa de escritura: se conservan con `stop` y se pierden con `down`; para un diagnóstico, míralos **antes** de recrear el contenedor
- Los registros de las aplicaciones YARN (MapReduce, Spark) se consultan con `yarn logs -applicationId ...`, como se ve en S1

Síntomas frecuentes

Síntoma	Qué mirar	Qué hacer
`Cannot connect to the Docker daemon`	`docker info`	Arrancar Docker Desktop o el servicio Docker
`port is already allocated`	`docker ps` (quién usa el puerto)	Parar ese contenedor o programa
`network hadoop-cluster ... not found`	`docker network ls`	`make -C entorno network` (lo hace `hadoop-up`)
Un contenedor en `Exited (137)`	`docker inspect ... OOMKilled`	Más memoria para Docker; parar servicios que no uses
Contenedores `running`, pero menos de 3 DataNodes en `dfsadmin -report`	`jps` y registro del DataNode	Esperar medio minuto; si sigue, leer `*-datanode-*.log`
`Incompatible clusterIDs` en un DataNode	`VERSION` de los volúmenes	Los volúmenes HDFS no son del mismo formateo: se borraron por separado
Faltan datos de una sesión anterior	`hdfs dfs -ls /datalake/...`	Repetir el notebook de esa sesión

Actividades

Actividades sin arrancar el clúster

- Con `docker image inspect`, averigua el `ENTRYPOINT`, la `CMD`, las variables de entorno y los puertos de `namenode-image` y `datanode-image`. ¿Qué diferencias hay y de qué línea del Dockerfile sale cada una?
- Con `docker image history`, localiza la capa del entorno Python del NameNode. ¿Cuánto ocupa frente a la de Hadoop? ¿Por qué no está en la imagen de los DataNodes?
- Abre una *shell* en un contenedor temporal de `datanode-image` (`docker run --rm -it ... bash`) y lee `/inicio.sh` y `yarn-site.xml`. ¿Qué memoria y cuántos vcores anuncia el NodeManager?
- Ejecuta `make -n -C entorno clean` y explica cada línea: ¿qué volúmenes sobreviven y por qué el Compose de Hadoop no lleva `-v`?
- Con `docker compose ... config`, comprueba qué valor final tienen `mem_limit` y `cpus`, y cambia la imagen del NameNode con la variable `NAMENODE_IMAGE` **sin** arrancar nada

Actividades con el clúster en marcha

- Desde `datanode1`, resuelve `namenode`, `resourcemanager` y `timelineserver`. ¿Por qué dan la misma dirección? ¿Qué cambia tras un `down` y un `hadoop-up`?
- Ejecuta `docker diff namenode` antes y después de lanzar un trabajo MapReduce. Predice qué se perdería con `make -C entorno down` y compruébalo (los datos de HDFS deben seguir ahí)
- Localiza en el volumen de `datanode1` los ficheros `blk_*` de un fichero tuyo y relaciónalos con `hdfs fsck <ruta> -files -blocks -locations`
- Para sólo un DataNode con `docker stop datanode3`. Observa con `hdfs dfsadmin -report` cuánto tarda en figurar como muerto y qué pasa con la replicación; vuelve a arrancarlo con `docker start datanode3`
- Mide con `docker stats` el consumo de `namenode` con y sin Jupyter arrancado, y compáralo con su `mem_limit`

Más allá de un equipo

Del laboratorio a un clúster real

- Compose gestiona contenedores en **un solo** equipo: el clúster del curso simula cuatro nodos que en realidad comparten la misma máquina
- En varios equipos, los contenedores los reparte un **orquestador**:
  - **Kubernetes**: el estándar de facto; Spark, Trino o Flink tienen modos de despliegue propios sobre él
  - **Docker Swarm**: modo de Docker que despliega un fichero Compose en varios nodos con `docker stack deploy`
- En la nube, servicios gestionados como AWS EMR ofrecen Hadoop y Spark ya instalados
- Las ideas se mantienen: imágenes inmutables, estado en volúmenes, servicios que se encuentran por nombre y configuración declarativa

Referencias

Documentación oficial

- Docker: *Dockerfile reference*, https://docs.docker.com/reference/dockerfile/
- Docker: *Volumes*, https://docs.docker.com/engine/storage/volumes/
- Docker: *Networking overview*, https://docs.docker.com/engine/network/
- Docker: *Compose file reference*, https://docs.docker.com/reference/compose-file/
- Docker: *Docker CLI reference*, https://docs.docker.com/reference/cli/docker/

Material del curso

- S1, «Cómo se construyen las imágenes del curso» y «Detener, limpiar y recuperar el entorno»
- `entorno/README.md`: órdenes del `Makefile` y solución de problemas